Computer Science & Engineering Department

# Arhitectura Sistemelor de Calcul

Curs 8
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



# Cuprins

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters



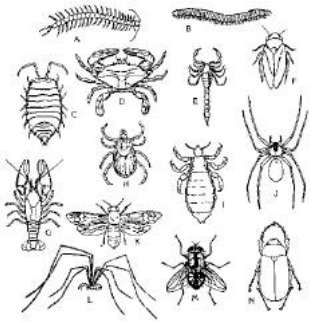





# Debugging and Performance Evaluation

## Debugging and Performance Evaluation

- Common errors in parallel programs
- Debugging tools
- Overview of benchmarking and performance measurements

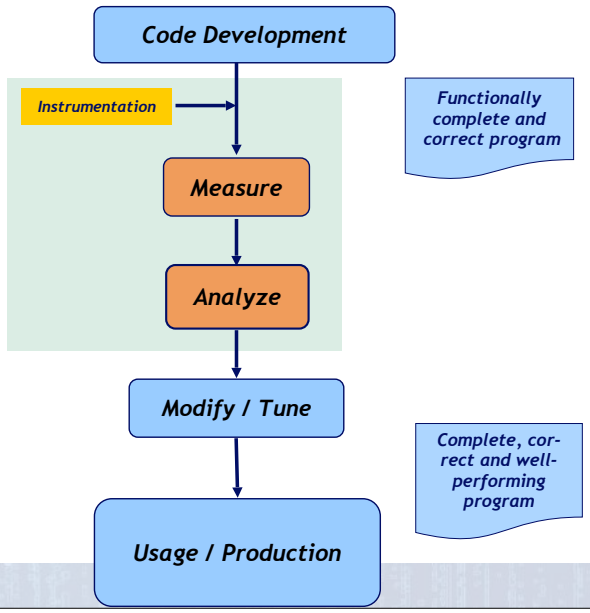






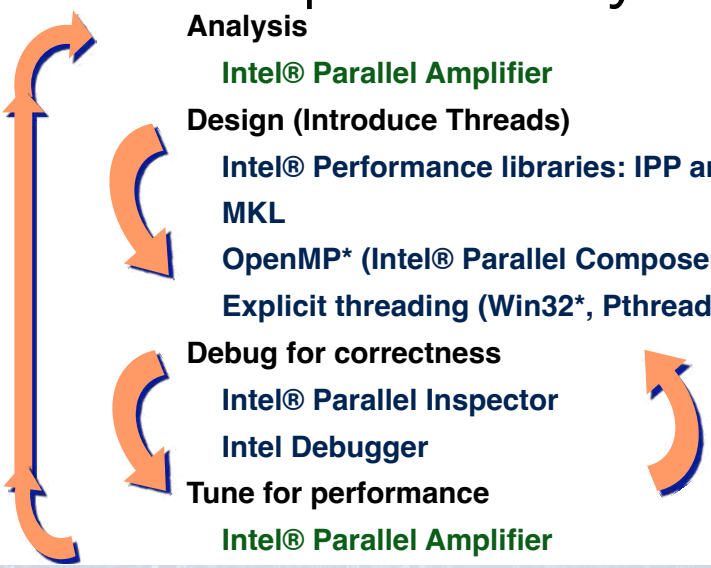
# Concepts and Definitions

- The typical performance optimization cycle






# Development Cycle



# Parallel Code Development

- Decide where to add the parallelism
  - Analyze the serial program
  - Prepare it for parallelism
- Add the parallelism
  - Threads, CUDA, OpenMP, Cilk, TBB, etc.
- Find logic problems
  - Only fails sometimes
  - Place of failure changes
- Find performance problems





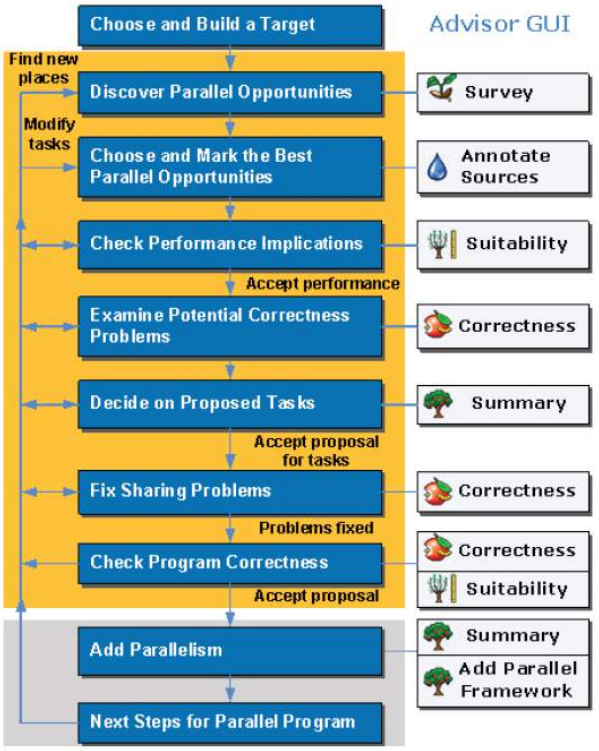
# Workflow

- Transforming many serial algorithms into parallel form takes five easy high-level steps
- Often existing algorithms are over-constrained by serial language semantics, and the underlying mathematics has a natural parallel expression if you can just find it



# Advisor Overview

- If you look at these steps in more detail, you find decisions you will need to make
- You do not have to choose the perfect answer the first time, so you can go back and modify your choices





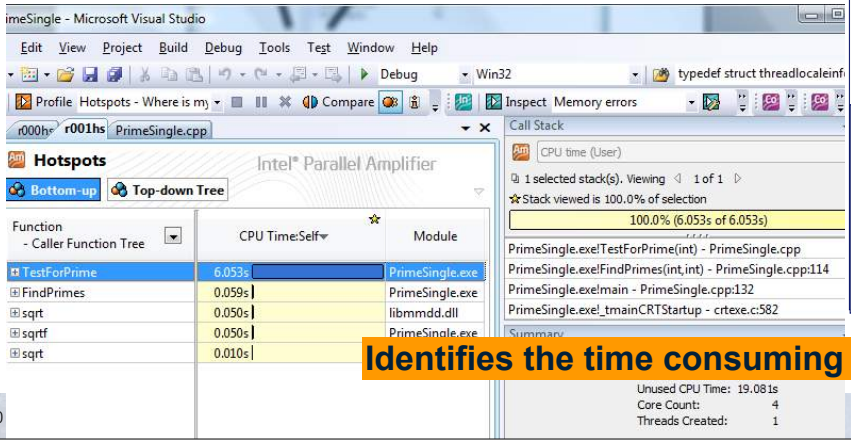

# Hotspot Analysis

- Use Parallel Amplifier to find hotspots in application

```
bool TestForPrime(int val)
{   // let's start checking from 3
    int limit, factor = 3;
    limit = (long)(sqrtf((float)val)+0.5f);
    while( (factor <= limit) && (val % factor))
            factor ++;

    return (factor > limit);
}
```

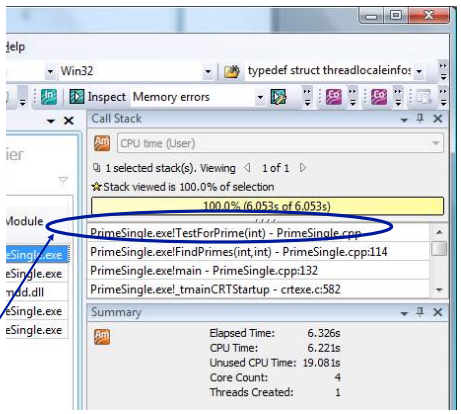
```
void FindPrimes(int start, int end)
{
    // start is always odd
    int range = end - start + 1;
    for( int i = start; i <= end; i+= 2 ){
        if( TestForPrime(i) )
            globalPrimes[gPrimesFound++] = i;
        ShowProgress(i, range);
    }
}
```




**Identifies the time consuming regions**

# Analysis - Call Stack

- Inspect the code for the leaf node
- Look for a loop to parallelize
  - If none are found, progress up the call stack until you find a suitable loop or function call to parallelize (FindPrimes)



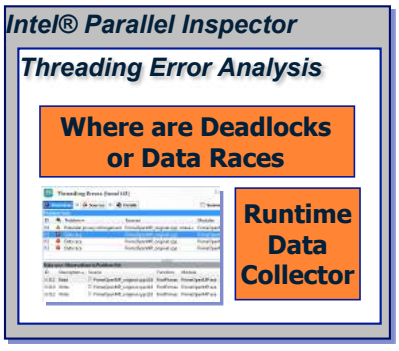

This is the level in the call tree where we need to thread

**Used to find proper level in the call-tree to thread**



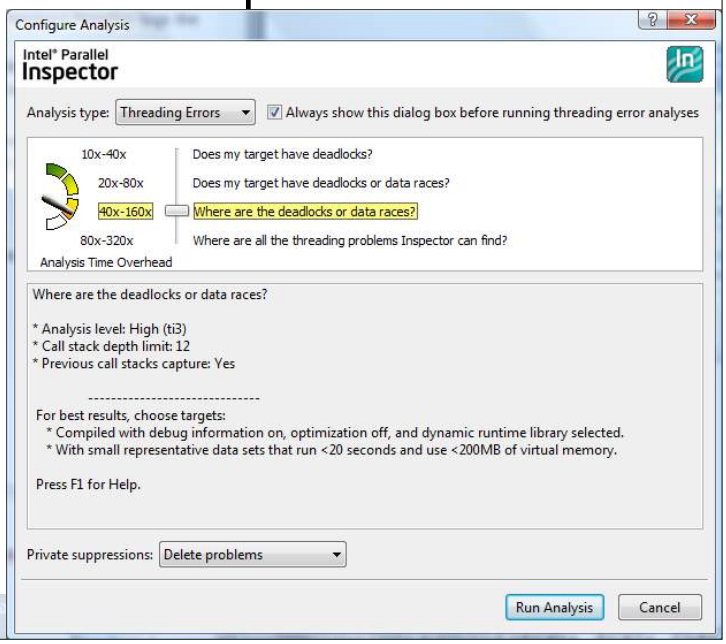
# Debugging for Correctness

- Intel® Parallel Inspector pinpoints notorious threading bugs like data races and deadlocks



# Intel® Parallel Inspector

- Select info regarding both data races & deadlocks
- View the Overview for Threading Errors
- Select a threading error and inspect the code






# Motivation

- Developing threaded applications can be a complex task
- New class of problems are caused by the interaction between concurrent threads
  - Data races or storage conflicts
    - More than one thread accesses memory without synchronization
  - Deadlocks
    - Thread waits for an event that will never happen

# Debugging Parallel Codes

- Debugging tool for threaded software
  - Plug-in to Microsoft* Visual Studio*
- Finds threading bugs in OpenMP*, Intel® Threading Building Blocks, and Win32* threaded software
- Locates bugs quickly that can take days to find using traditional methods and tools
  - Isolates **problems**, not the **symptoms**
  - **Bug does not have to occur to find it!**





# Parallel Inspector: Analysis

- Dynamic as software runs
  - Data (workload)-driven execution
- Includes monitoring of:
  - Thread and Sync APIs used
  - Thread execution order
    - Scheduler impacts results
  - Memory accesses between threads



**Code path must be executed to be analyzed**



# Parallel Inspector: Before You Start

- Instrumentation: background
  - Adds calls to library to record information
    - Thread and Sync APIs
    - Memory accesses
  - Increases execution **time** and **size**
- Use **small** data sets (workloads)
  - Execution time and space is **expanded**
  - Multiple runs over different paths yield best results



**Workload selection is important!**



# Workload Guidelines

- Execute problem code once per thread to be identified
- Use smallest possible working data set
  - Minimize data set size
    - Smaller image sizes
  - Minimize loop iterations or time steps
    - Simulate minutes rather than days
  - Minimize update rates
    - Lower frames per second



**Finds threading errors faster!**



# Binary Instrumentation

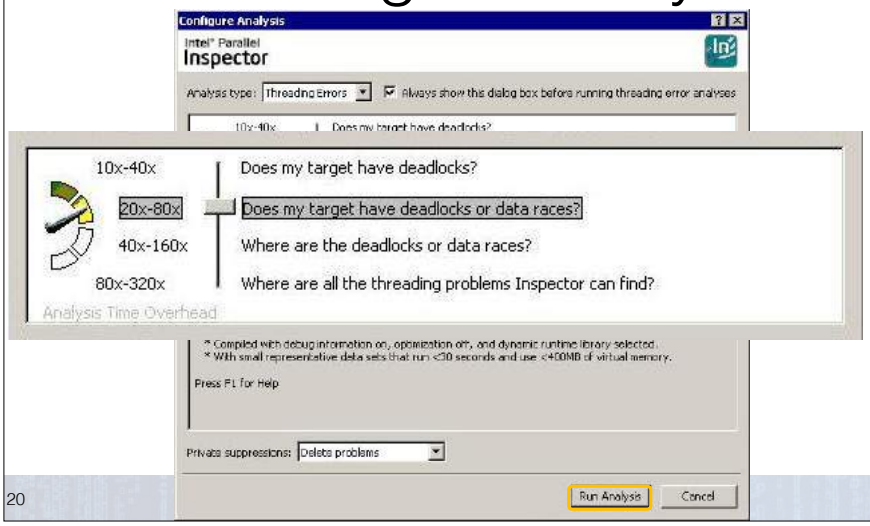
- Build with supported compiler
- Running the application
  - Must be run from within Parallel Inspector
  - Application is instrumented when executed
  - External DLLs are instrumented as used





# Parallel Inspector Startup

- The Configure Analysis window pops up




Select the level of analysis to be carried out by Parallel Inspector

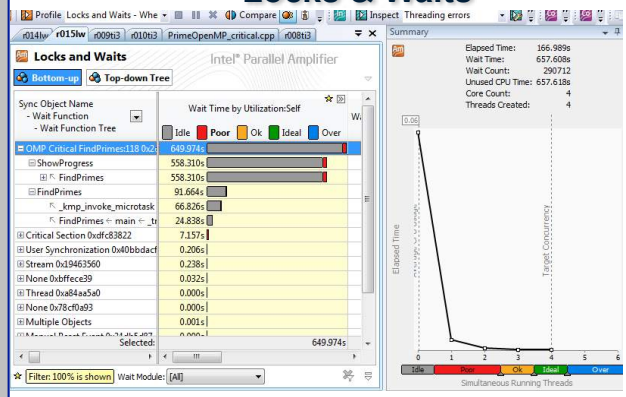
**The deeper the analysis, the more thorough the results and the longer the execution time**

Click Run Analysis

# Tuning for Performance

**Intel® Parallel Amplifier**

**Locks & Waits**

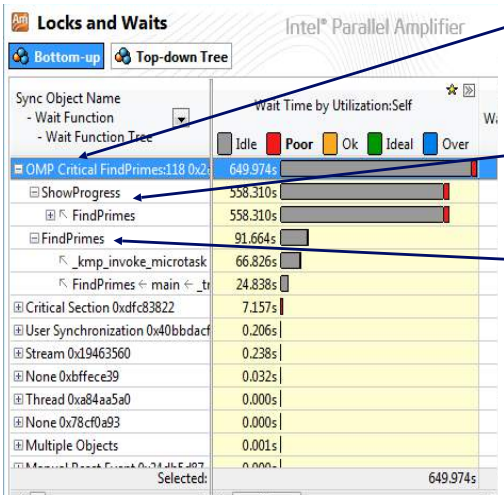



- Parallel Amplifier (Locks & Waits) pinpoints performance bottlenecks in threaded applications





# Parallel Amplifier - Locks & Waits



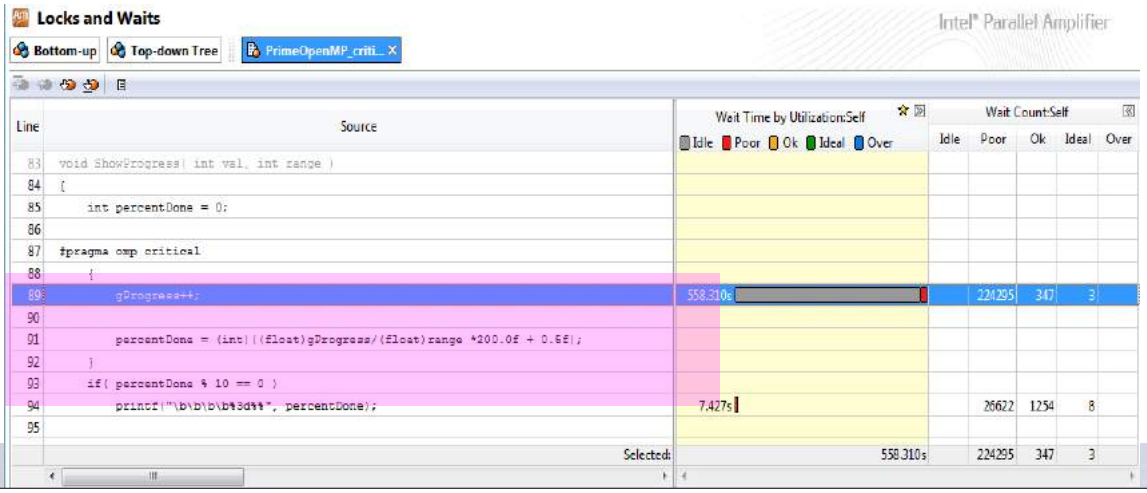

- Graph shows significant portion of time in idle condition as result of a critical section
- FindPrimes() & ShowProgress() are both excessively impacted by the idle time occurring in the critical section





# Parallel Amplifier - Locks & Waits

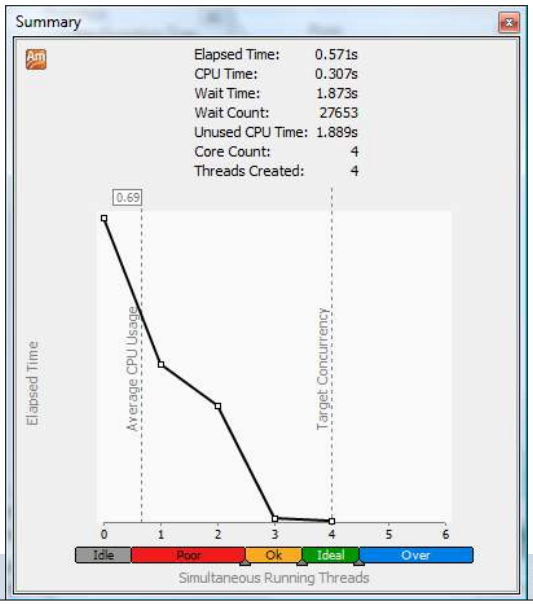
- ShowProgress() consumes 558/657 (85%) of the time idling in a critical section
- Double Click ShowProgress() in largest critical section to see the source code





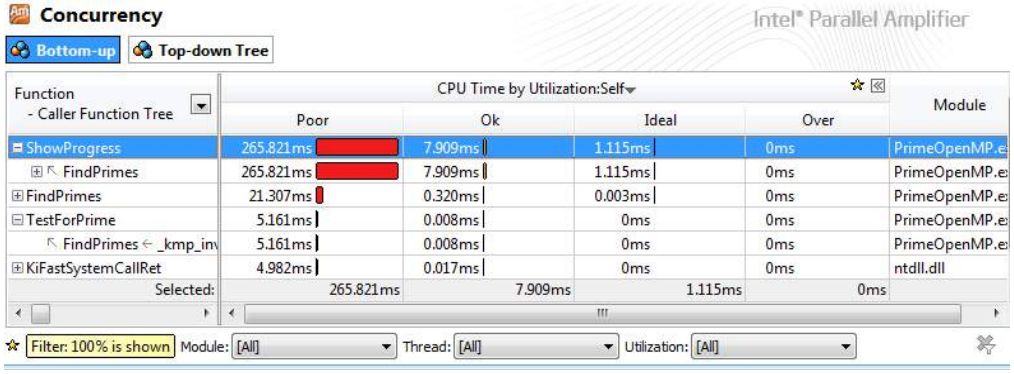

# Parallel Amplifier Summary

- Elapsed Time shows 0.571 sec
- Wait Time/ Core Count:
  - 1.87/4 =0.47 sec
- Waiting 82% of elapsed time in critical section
- Most of the time 1 core and occasionally 2 are occupied



# Parallel Amplifier - Concurrency

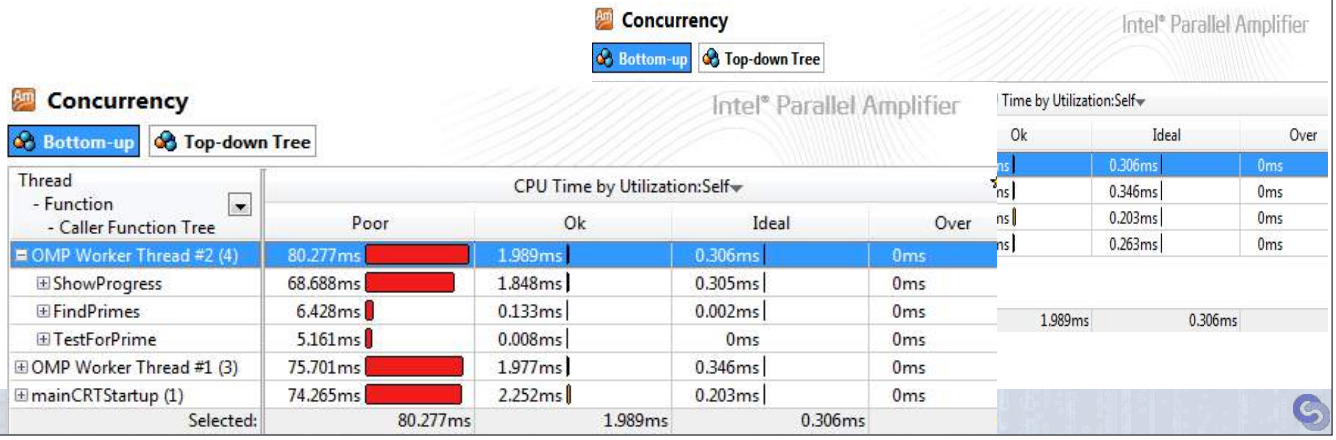
- Function - Caller Function Tree
- ShowProgress is called from FindPrimes and represent the biggest reason concurrency is poor





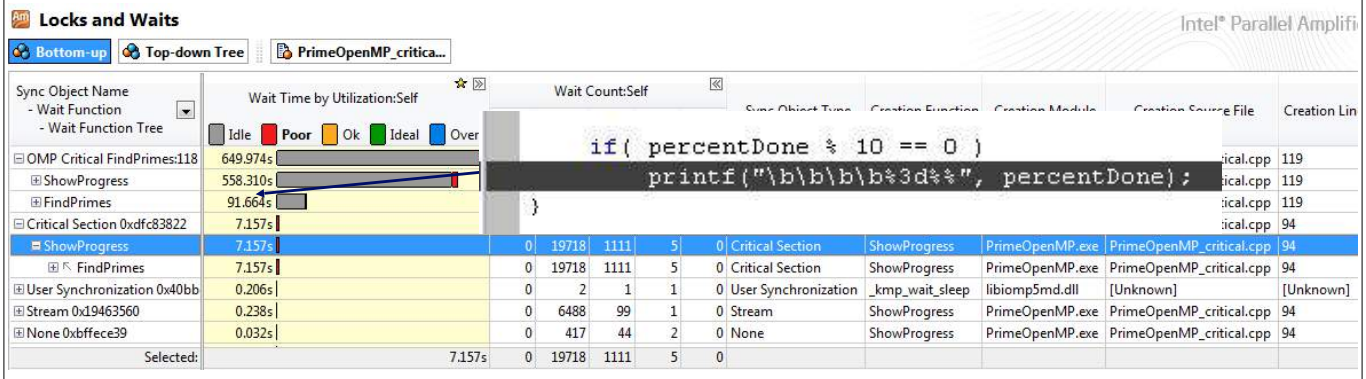

# Parallel Amplifier - Concurrency

- Thread –Function –Caller Function Tree
  - This view shows how each thread contributes to the concurrency issue
- Expanding any thread will reveal the functions that contribute most



# Performance

- Double Click ShowProgress in second largest critical section
- This implementation has implicit synchronization calls - printf
- This limits scaling performance due to the resulting context switches

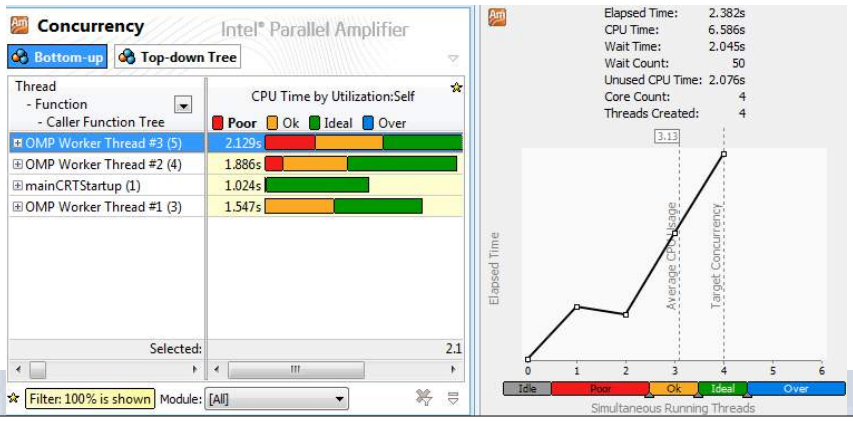



**Back to the design stage**



# Load Balance Analysis

- Use Parallel Amplifier – Concurrency Analysis
- Select the “Thread – Function – Caller Function Tree”
- Observe that the 4 threads do unequal amounts of work



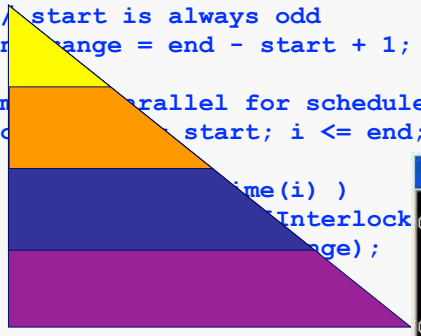
# Fixing the Load Imbalance

- Distribute the work more evenly

```
void FindPrimes(int start, int end)
{
    // start is always odd
    int range = end - start + 1;

#pragma parallel for schedule(sta
    for (int i = start; i <= end; i +=
    {
        ime(i) )
        Interlock
        nge);

    }
}
```



```
C:\WINDOWS\system32\cmd.exe

C:\classfiles\PrimeOpenMP\Debug>PrimeOpenMP.exe 1 5000000
 90%

  348513 primes found between      1 and 5000000 in     4.22 secs

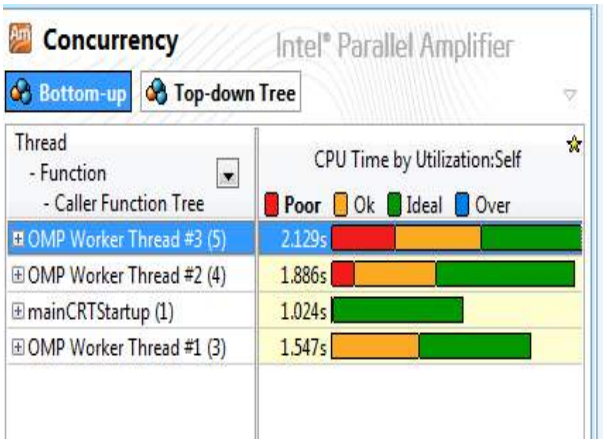
C:\classfiles\PrimeOpenMP\Debug>_
```

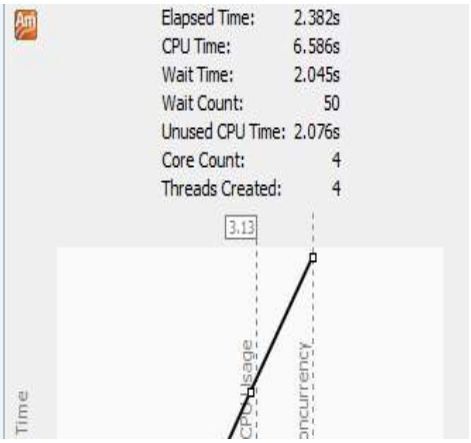
**Speedup achieved is 1.68X**





# Comparative Analysis




**Threading applications require multiple iterations of going through the software development cycle**







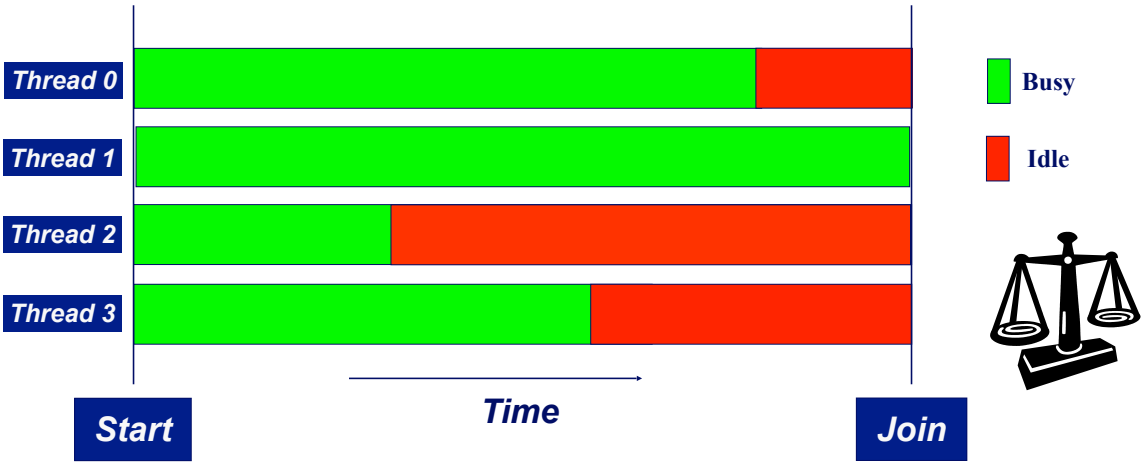

# Performance Issues

# Common Performance Issues

- **Load balance**
  - Improper distribution of parallel work
- **Synchronization**
  - Excessive use of global data / contention for same synchronization object
- Parallel Overhead
  - Due to thread creation, scheduling
- Granularity
  - Not sufficient parallel work

# Load Imbalance

- Unequal work loads lead to idle threads and wasted time






# Redistribute Work to Threads

- Static assignment
  - Are the same number of tasks assigned to each thread?
  - Do tasks take different processing time?
    - Do tasks change in a predictable pattern?
    - Rearrange (static) order of assignment to threads
    - Use dynamic assignment of tasks

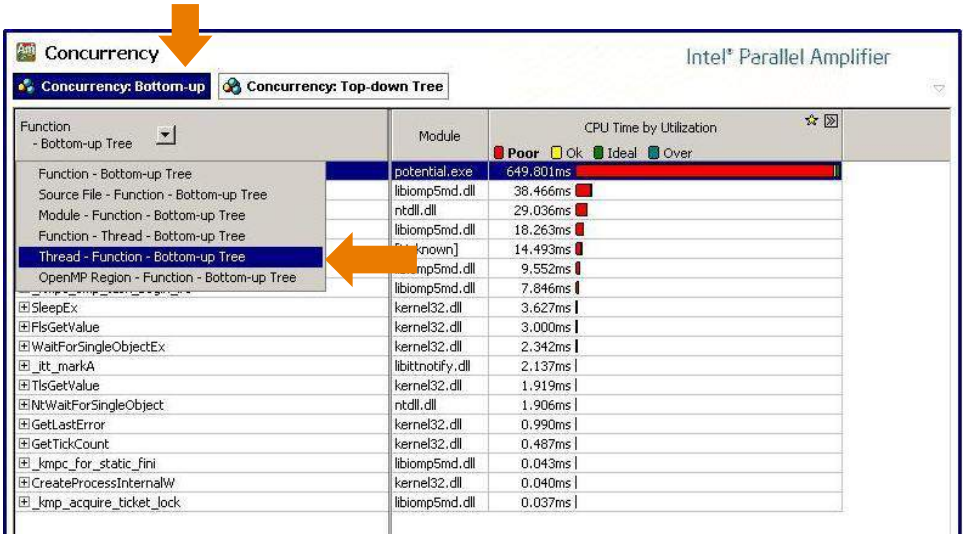
# Redistribute Work to Threads

- Dynamic assignment
  - Is there one big task being assigned?
    - Break up large task to smaller parts
  - Are small computations agglomerated into larger task?
    - Adjust number of computations in a task?
    - More small computations into single task?
    - Fewer small computations into single task?





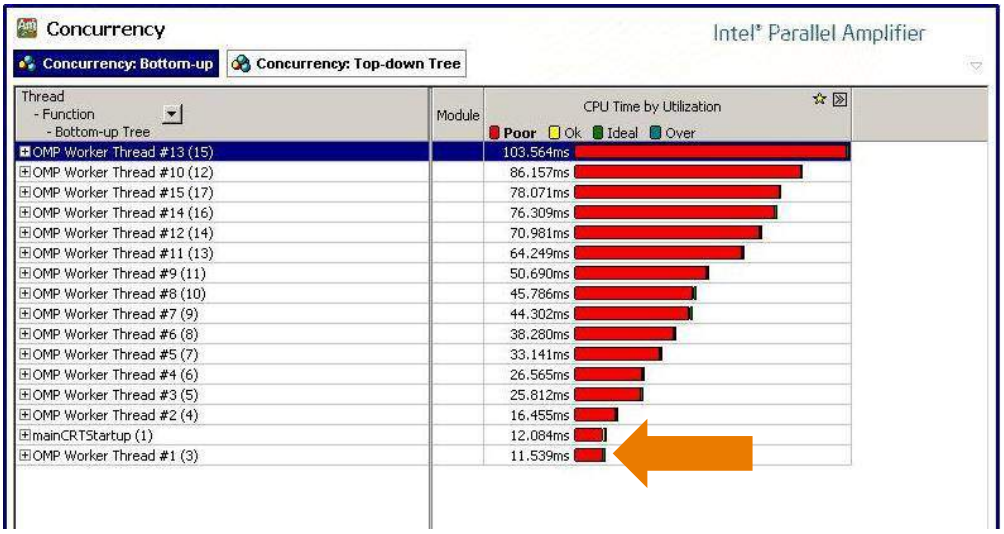
# Unbalanced Workloads







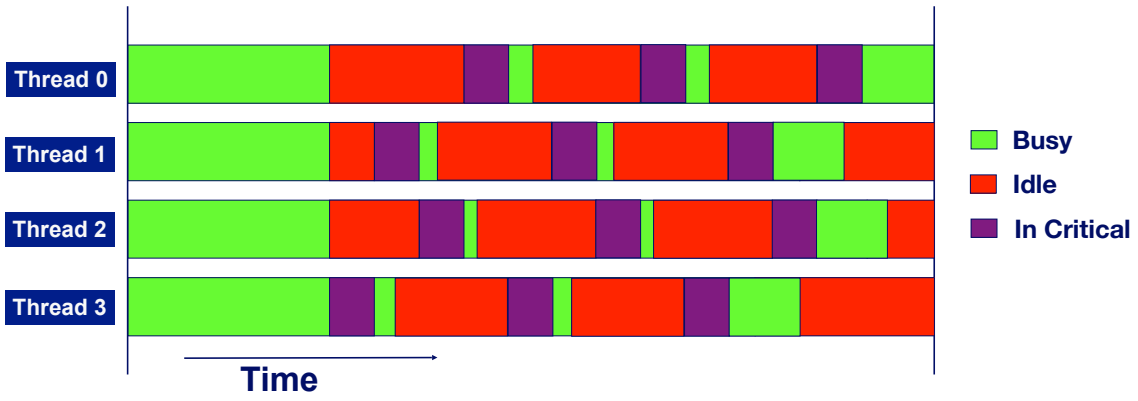

# Unbalanced Workloads






# Synchronization

- By definition, synchronization serializes execution
- Lock contention means more idle time for threads



# Synchronization Fixes

- Eliminate synchronization
  - Expensive but necessary “evil”
  - Use storage local to threads
    - Use local variable for partial results, update global after local computations
    - Allocate space on thread stack (**alloca**)
    - Use thread-local storage API (**TlsAlloc**)
  - Use atomic updates whenever possible
    - Some global data updates can use atomic operations





# General Optimizations

- Serial Optimizations
  - Serial optimizations along the critical path should affect execution time
- Parallel Optimizations
  - Reduce synchronization object contention
  - Balance workload
  - Functional parallelism
- Analyze benefit of increasing number of processors
- Analyze the effect of increasing the number of threads on scaling performance







# Profiling and Tracing

# Measurement: Profiling vs. Tracing

- Profiling
  - Summary statistics of performance metrics
    - Number of times a routine was invoked
    - Exclusive & inclusive time
    - Hardware performance counters
    - Number of child routines invoked, etc.
    - Structure of invocations (call-trees/call-graphs)
    - Memory, message communication sizes
- Tracing
  - When and where events took place along a global timeline
    - Time-stamped log of events
    - Message communication events (sends/receives) are tracked
    - Shows when and from/to where messages were sent
    - Large volume of performance data generated leads to more perturbation in the program





# Measurement: Profiling

- Profiling
  - Helps to expose performance bottlenecks and hotspots
  - 80/20 – rule or Pareto principle: often 80% of the execution time in 20% of your application
  - Optimize what matters, don't waste time optimizing things that have negligible overall influence on performance
- Implementation
  - Sampling: periodic OS interrupts or hardware counter traps
    - Build a histogram of sampled program counter (PC) values
    - Hotspots will show up as regions with many hits
  - Measurement: direct insertion of measurement code
    - Measure at start and end of regions of interests, compute difference





# Measurement: Tracing

- Tracing
  - Recording of information about significant points (events) during program execution
    - Entering/exiting code region (function, loop, block, …)
    - Thread/process interactions (e.g., send/receive message)
  - Save information in event record
    - Timestamp
    - CPU identifier, thread identifier
    - Event type and event-specific information
  - Event trace is a time-sequenced stream of event records
  - Can be used to reconstruct dynamic program behavior
  - Typically requires code instrumentation







# Performance Counters

# Hardware Performance Counters

- Specialized hardware registers to measure the performance of various aspects of a microprocessor
- Originally used for hardware verification purposes
- Can provide insight into:
  - Cache behavior
  - Branching behavior
  - Memory and resource contention and access patterns
  - Pipeline stalls
  - Floating point efficiency
  - Instructions per cycle
- Counters vs. events
  - Usually a large number of countable events - hundreds
  - On a small number of counters (4-18)





# What is PAPI

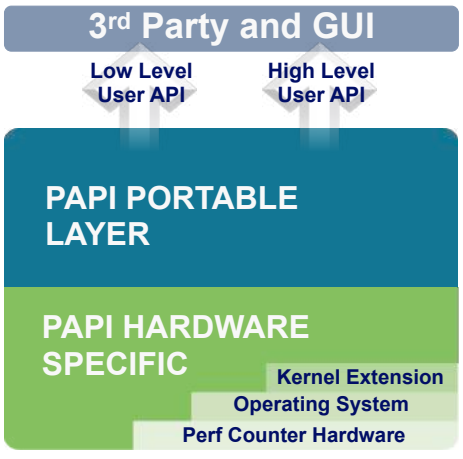
- Middleware that provides a consistent and efficient programming interface for the performance counter hardware found in most major microprocessors.
- Countable events are defined in two ways:
  - Platform-neutral Preset Events (e.g., PAPI_TOT_INS)
  - Platform-dependent Native Events (e.g., L3_CACHE_MISS)
- Preset Events can be derived from multiple Native Events
  - PAPI_L1_TCM (Total Cache Misses) **might** L1 Data Misses + L1 Instruction Misses
- Preset events are defined in a best-effort way
  - No guarantees of semantics portably
  - Figuring out what a counter actually counts and if it does so correctly can be hairy





# PAPI Counter Interfaces

- PAPI provides 3 interfaces to the underlying counter hardware:
  - A low level API manages **hardware events** in user defined groups called EventSets.
  - A high level API provides the ability to **start**, **stop** and **read** the **counters** for a specified list of events.
  - Graphical and end-user tools provide facile data collection and visualization.








# PAPI Example Low Level API Usage

```
#include "papi.h"
#define NUM_EVENTS 2
int Events[NUM_EVENTS]={PAPI_FP_OPS,PAPI_TOT_CYC},
int EventSet;
long long values[NUM_EVENTS];
/* Initialize the Library */
retval = PAPI_library_init (PAPI_VER_CURRENT);
/* Allocate space for the new eventset and do setup */
retval = PAPI_create_eventset (&EventSet);
/* Add Flops and total cycles to the eventset */
retval = PAPI_add_events (&EventSet,Events,NUM_EVENTS);
/* Start the counters */
retval = PAPI_start (EventSet);

do_work();  /* What we want to monitor*/

/*Stop counters and store results in values */
retval = PAPI_stop (EventSet,values);
```





# Using PAPI Through Tools

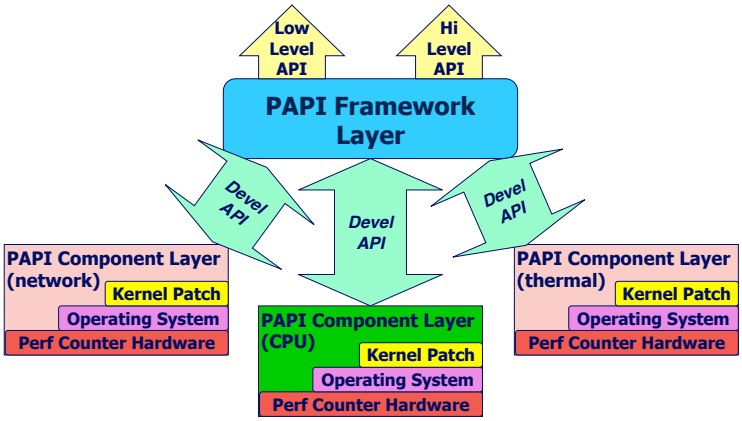
- You can use PAPI directly in your application, but most people use it through tools
- The tool might have a predefined set of counters or lets you select counters through a configuration file/environment variable, etc.
- Tools using PAPI
  - Scalasca, KOJAK (FZ Juelich, UTK)
  - TAU (UO)
  - PerfSuite (NCSA)
  - HPCToolkit (Rice)
  - Open|Speedshop (SGI)
  - ompP (UCB)
  - IPM (LBNL)





# Component PAPI Design

**Re-Implementation of PAPI w/ support for multiple monitoring domains (CPU, network, thermal)**








# Ce am invatat in aces curs?

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters





Thank you for your attention.







emil.slusanschi@cs.pub.ro
cs.pub.ro